

MODULE 2

AI-Augmented Development

Verification, Governance & the Quality Gate

Renaissance Developer Academy — Detailed Lesson Plan

Week 2 of 10 | 5 Days | Full-Time Intensive
Badge Earned: AI-Augmented Developer

Prerequisite: Module 1 — Renaissance Foundations
DRAFT — March 2026

Module Overview

In Module 1, students met the Renaissance Developer archetype and got their first taste of AI-augmented building. Module 2 goes deeper: this week is about becoming a **critical consumer** of AI-generated code, not just a fast producer. The central insight of the week is that **reviewing AI output is harder than writing code yourself** — and that the Quality Gate from the AI Multiplier Model (AI Generation → Human Verification → Business Value) is where the Renaissance Developer earns their keep.

The week builds on students' Module 1 experience with Nanobot (where they forked, read, and extended the codebase) and their initial AI tool proficiency. Now we move from “can I use these tools?” to “can I trust what they produce, systematically verify it, and teach others to do the same?”

Connection to the JD: This module maps directly to the Practitioner-level skills in AI Evaluation & Verification, AI Literacy, and AI-Augmented Development. Students practice creating evaluation standards, designing review processes for AI-assisted work, and balancing AI speed with verification rigor.

Learning Objectives

1. Master systematic prompt engineering: repeatable patterns for code generation, debugging, refactoring, and documentation
2. Build a calibrated AI failure detector: identify subtle bugs, security vulnerabilities, performance issues, and logic errors in AI-generated code
3. Design AI code review workflows with structured checklists that scale across a team
4. Extend autonomous agents with multi-tool capabilities and persistent memory systems
5. Create an AI governance framework: evaluation standards, verification checklists, and human-in-the-loop workflow designs
6. Articulate the costs and trade-offs of AI-augmented development: when AI helps, when it hurts, and when to turn it off

Pre-Work (Assigned End of Module 1)

Due before Day 1. Estimated time: 2–3 hours.

1. **NotebookLM Audio Overview:** Listen to the Module 2 Deep Dive audio you generated on Module 1 Day 5. Come with 3 questions.
2. **Read:** “The Product-Minded Engineer” by Gergely Orosz (article). Note: how does product thinking connect to AI verification? If the AI builds the wrong thing fast, speed is a liability.
3. **Explore:** Use Claude Code CLI for at least 1 hour of real coding this weekend. Keep a running log of: (a) where AI was correct, (b) where AI was subtly wrong, (c) where AI was confidently wrong. Bring this log to Day 1.

4. **Prepare:** Review your Module 1 Nanobot fork and custom skill. Ensure it still runs locally. You'll build on it this week.

Week-at-a-Glance

Day	Theme	Primary Tools	Key Output
Day 1	Advanced Prompt Engineering & AI Code Generation	Claude Code, Kiro, Antigravity	Prompt playbook, generation experiments
Day 2	The AI Bug Safari — Training Your Failure Detector	Curated code samples, Claude Code	AI Bug Log (10 documented failures)
Day 3	AI Code Review Workflows & Verification Frameworks	GitHub PRs, all tools	AI Review Checklist, team workflow design
Day 4	Advanced Agent Development: Multi-Tool & Memory	Nanobot, Claude Code, GitHub Actions	Enhanced agent with 3+ tools, memory system
Day 5	AI Governance Lab, Reflection & Calibration	NotebookLM, all tools	Governance framework, development map update

Day 1: Advanced Prompt Engineering & AI Code Generation

Theme: Move from ad-hoc prompting to systematic, repeatable prompt patterns that produce verifiable code

Goal: Every student has a structured prompt playbook with 15+ battle-tested patterns, understands why prompt structure matters for verification, and can articulate the difference between prompting for speed vs. prompting for correctness.

Schedule

09:00	Week 2 Opening: From Generator to Verifier	30 min
	- Recap the AI Multiplier Model from Module 1: AI Generation → Human Verification → Business Value - This week's focus: the Quality Gate — the human verification step that makes or breaks AI-augmented work - Share weekend coding logs from pre-work: patterns in where AI was subtly wrong - Key framing: "Last week you learned to use AI tools. This week you learn to not trust them blindly."	
09:30	The Anatomy of an Effective Prompt (Concept Session)	60 min
	- Why prompt structure matters: predictable inputs → verifiable outputs → repeatable quality - The five layers of a well-structured prompt: — Layer 1: Context — project background, tech stack, coding standards, existing architecture — Layer 2: Task — specific, bounded request with clear acceptance criteria — Layer 3: Constraints — language, framework, testing requirements, security policies, performance targets — Layer 4: Output format — file structure, naming conventions, documentation expectations — Layer 5: Verification hooks — asking the AI to explain its reasoning, flag uncertainty, suggest tests - Live demo: the same task prompted at Layer 1 only vs. all 5 layers — compare output quality - Key insight: the more structured your prompt, the easier the verification. Sloppy prompts create unverifiable code.	
10:30	Break	15 min
	Questions	
10:45	Prompt Pattern Workshop	90 min

	• EXERCISE 1: Build your prompt playbook — systematic patterns for common tasks: • Pattern 1 — Scaffolding: generate a new module/service/component with full structure • — Practice: scaffold a REST API service with routes, controllers, models, and tests • — Include: project context, file naming convention, error handling pattern, test framework choice • Pattern 2 — Test Generation: produce comprehensive tests for existing code • — Practice: give AI your Module 1 Nanobot custom skill, ask for tests • — Verify: are edge cases covered? Are tests actually testing behavior or just mocking everything? • Pattern 3 — Refactoring: transform working code into better code without changing behavior • — Practice: take a messy function and ask for refactoring with specific goals (readability, performance, testability) • — Key question: did the refactoring preserve behavior? How would you verify? • Pattern 4 — Debugging: use AI to diagnose and fix issues • — Practice: give AI a failing test output and ask it to identify the root cause • — Anti-pattern: AI “fixing” bugs by changing the test to pass instead of fixing the code • Pattern 5 — Documentation: generate docs, READMEs, and inline comments from code • — Practice: generate API documentation for your Module 1 Day 4 app • — Verify: are the docs accurate? Do they match the actual behavior? • Commit all patterns to your prompts/ directory with examples and notes
--	---

12:15	Lunch	60 min
	• Optional: prompt pattern sharing session	

13:15	Chain-of-Thought & Multi-Step Prompting	60 min
	• Beyond single-shot prompts: orchestrating AI across multi-step workflows • Chain-of-thought prompting: asking AI to reason before coding • — “Before writing code, analyze the requirements and identify 3 potential design approaches. For each, explain trade-offs. Then implement the best one.” • — Why this works: forces the AI to make its reasoning explicit → easier to verify • Decomposition prompting: breaking complex tasks into verifiable subtasks • — Instead of: “Build an authentication system” • — Try: “(1) Design the data model for users and sessions. (2) Implement password hashing. (3) Build the login endpoint. (4) Add session management. (5) Write integration tests.” • — Each step is independently verifiable — connecting to the Precision mindset • EXERCISE 2: Take one complex feature from your Module 1 Day 4 app and rebuild it using: • — (a) Single-shot prompt — time it, assess quality	

	• — (b) Chain-of-thought prompt — time it, assess quality • — (c) Decomposition prompt — time it, assess quality • — Document: which produced the best code? Which was easiest to verify? Which was fastest?
--	--

14:15	Break	15 min
	• Stretch	

14:30	Prompt Engineering Across Tools: Comparative Lab	75 min
	• EXERCISE 3: The Prompt Portability Test • Take your 3 best prompt patterns and use them with all three AI tools: • — Claude Code CLI: terminal-native, conversational, context-aware across session • — Kiro: spec-driven — how do prompts translate into requirements/design/tasks? • — Antigravity: agent-assisted — how do prompts work when the agent has autonomy? • Key questions to document: • — Does the same prompt produce the same quality across tools? Why or why not? • — Which tools benefit most from highly structured prompts? • — Which tools compensate for vague prompts with their own structure (Kiro's spec workflow)? • — How does context window size affect prompt strategy? • Write up findings in your repo wiki — this becomes part of your prompt playbook	

15:45	Pair Exercise: Prompt Review	45 min
	• EXERCISE 4: Exchange prompt playbooks with a partner • — Each person tries their partner's prompts on a new codebase • — Do the prompts work for someone who didn't write them? Where do they break? • — Provide feedback: which patterns are great, which need more context or constraints • — Key insight: prompts that only work for you are not team-scalable — the Renaissance Developer creates practices that work across teams	

16:30	Day 1 Wrap-Up	30 min
	• Share: one prompt pattern that surprised you with how much it improved output quality • HOMEWORK: Add 5 more patterns to your playbook (total 15+ patterns). Include at least 1 chain-of-thought and 1 decomposition pattern • HOMEWORK: Generate 3 pieces of AI code tonight that you suspect contain subtle bugs. Save them for tomorrow's Bug Safari • Preview: tomorrow we train your failure detector — the AI Bug Safari	

Instructor Notes — Day 1: *The five-layer prompt structure is the day's anchor concept. Students who internalize it will produce verifiable code for the rest of the program. Watch for two anti-patterns: (1) students who write prompts that are TOO long and prescriptive, removing the AI's ability to contribute — coach them toward constraints that guide rather than dictate; (2) students who rush through the comparative lab (Exercise 3) without documenting findings — the documentation IS the learning. The pair review (Exercise 4) often reveals that experienced developers write prompts assuming context their partner doesn't have — this is the first lesson in writing team-scalable AI practices.*

Day 2: The AI Bug Safari — Training Your Failure Detector

Theme: Develop a calibrated intuition for where AI-generated code fails, including subtle bugs that pass tests but contain logic errors, security vulnerabilities, or performance issues

Goal: Every student completes an AI Bug Log with 10 documented failures, each with root cause analysis and verification strategy. This log becomes a personal reference and a team training resource.

Schedule

09:00	The Failure Taxonomy (Concept Session)	45 min
	- Why AI fails: understanding the patterns behind AI code errors - Category 1 — Confident Hallucination: AI invents APIs, functions, or libraries that don't exist — Example: calling a method that sounds plausible but was never defined - Category 2 — Subtle Logic Errors: code that looks correct and passes basic tests but handles edge cases wrong — Example: off-by-one errors, incorrect null handling, wrong operator precedence - Category 3 — Security Blind Spots: AI generates code that works but is vulnerable — Example: SQL injection, insecure defaults, hardcoded secrets, missing input validation - Category 4 — Performance Anti-Patterns: functionally correct code that scales terribly — Example: N+1 queries, unnecessary memory copies, missing indexes, synchronous blocking - Category 5 — Stale Knowledge: AI uses deprecated APIs, old patterns, or outdated best practices — Example: using callback patterns in modern async/await code, referencing removed library features - Category 6 — Copy Without Context: AI clones patterns without adapting to the specific codebase — Example: the 4x code clone problem — identical error handling patterns copy-pasted regardless of context - Industry data: GitClear found 4x growth in code duplication since AI coding tools became widespread	
09:45	The AI Bug Safari: Round 1 — Planted Bugs	90 min
	- EXERCISE 5: The Bug Safari (instructor-curated samples) - You will review 10 code samples, each AI-generated, each containing 1–3 bugs across the six categories: — Sample 1: A user authentication function (Category 3 — security: timing attack vulnerability)	

	— Sample 2: A data processing pipeline (Category 4 — performance: $O(n^2)$ hidden in list comprehension) — Sample 3: A REST API endpoint (Category 2 — logic: incorrect HTTP status codes for edge cases) — Sample 4: A database query builder (Category 3 — security: SQL injection via string interpolation) — Sample 5: A file upload handler (Category 2 — logic: race condition in concurrent uploads) — Sample 6: A caching layer (Category 4 — performance: cache invalidation bug causing stale data) — Sample 7: A WebSocket handler (Category 1 — hallucination: uses a non-existent API method) — Sample 8: A configuration parser (Category 6 — copy: generic error handling ignoring config-specific needs) — Sample 9: A rate limiter (Category 5 — stale: uses deprecated token bucket implementation) — Sample 10: A test suite (Category 2 — logic: tests that pass but don't actually test the stated behavior) - For each sample, document: (a) what bugs you found, (b) which category, (c) why AI made this error, (d) how you would verify/catch it - Work individually first (45 min), then compare findings with a partner (30 min) - Scoring: instructor reveals the planted bugs after pair discussion — how many did you find?
--	---

11:15	Break	15 min
	Decompress — bug hunting is mentally taxing	

11:30	Safari Debrief & Pattern Recognition	45 min
	- Instructor-led debrief: walk through every planted bug and the intended lesson - Discussion: which categories were hardest to spot? Why? - Common finding: security bugs are spotted by <30% of engineers on first pass - Common finding: performance bugs are nearly invisible without load testing or Big-O analysis - Key insight: AI bugs are not random — they follow patterns. Once you learn the patterns, you see them everywhere. - Introduce the “Verification Lens”: a mental checklist for every AI-generated code block: — Does this do what I asked? (correctness) — Does this handle what I didn't ask? (edge cases) — Would I be comfortable if this ran with real user data? (security) — Would I be comfortable if this ran with 10x traffic? (performance) — Will someone else understand this in 6 months? (maintainability)	

12:15	Lunch	60 min
--------------	--------------	--------

	• Informal: share your most embarrassing AI bug story from real work
13:15	The AI Bug Safari: Round 2 — Wild Bugs 90 min • EXERCISE 6: Generate your own wild bugs • Use Claude Code CLI, Kiro, or Antigravity to generate code for these real-world tasks: • — Task A: “Build a rate limiter middleware with sliding window algorithm” • — Task B: “Implement a JWT refresh token rotation system” • — Task C: “Create a data migration script that transforms user profiles from schema v1 to v2” • — Task D: “Build a webhook receiver with retry logic and idempotency” • For each generated solution: • — Apply the Verification Lens (correctness, edge cases, security, performance, maintainability) • — Document every bug or concern you find • — Classify using the six-category taxonomy • — Write the test or verification strategy that would catch this bug • Goal: find at least 5 more bugs to bring your AI Bug Log total to 10+ • Pair verification: swap generated code with a partner — can they find bugs you missed?
14:45	Break 15 min • Stretch
15:00	AI Bug Log Workshop: Documentation as Learning Tool 60 min • EXERCISE 7: Compile your AI Bug Log • Structure each entry using this template (committed to your repo): • — Bug ID and title • — AI tool and prompt used to generate the code • — The buggy code (highlighted section) • — Bug category (from the taxonomy) • — Root cause: why did the AI produce this error? • — Detection method: how did you (or would you) catch it? • — Fix: the corrected code • — Prevention: what prompt improvement or verification step would prevent this in the future? • Minimum: 10 documented bugs (5 from Safari Round 1, 5+ from Round 2) • The Bug Log is not just an exercise — it’s a personal reference you’ll use throughout the program • Best practice: add to it throughout the program whenever you encounter a new AI failure pattern

16:00	Day 2 Wrap-Up	30 min
	• Bug Log Gallery Walk: post your 3 best bug entries on the class board, browse others' findings • Discuss: collectively, what are the top 5 failure patterns this cohort identified? • HOMEWORK: Add 2 more bugs from tonight's coding to your Bug Log (total 12+) • HOMEWORK: Draft a 1-page "AI Code Review Checklist" for your future team — based on the patterns you've seen • Preview: tomorrow we formalize this into review workflows and verification frameworks	

Instructor Notes — Day 2: *The Bug Safari is the signature exercise of Module 2 and one of the most talked-about experiences in the program. The 10 planted samples must be carefully curated — they should be realistic enough that students could believe they wrote the code, not obviously broken. Prepare answer keys with detailed explanations. During Round 1 scoring, celebrate rather than shame — missing security bugs is normal and the whole point of calibration. Round 2 (wild bugs) is where students build real intuition because the bugs are not planted — they're genuine AI failures. The Bug Log Gallery Walk at end-of-day creates communal learning from individual discoveries. Start a shared Bug Log repository that the cohort contributes to throughout the program.*

Day 3: AI Code Review Workflows & Verification Frameworks

Theme: Transform individual bug-spotting skills into scalable team workflows, checklists, and review processes

Goal: Every student has a published AI Code Review Checklist, has practiced structured AI-aware code review on real PRs, and has designed a team-scalable verification workflow.

Schedule

09:00	From Individual Intuition to Team Process (Concept Session)	45 min
	- The gap: you can spot AI bugs now, but can your team? Can someone who joins in 6 months? - Connecting to the JD: “Create evaluation standards and checklists, train engineers on rigorous AI verification” - Three levels of AI-aware code review: — Level 1: Personal discipline — your own Verification Lens (learned yesterday) — Level 2: Team process — structured checklists embedded in PR templates and CI/CD — Level 3: Organizational standards — automated checks, review rubrics, training programs - Case study: how code review changes when 70%+ of code is AI-generated — The reviewer must understand intent, not just implementation — Test coverage becomes a first-class verification tool, not an afterthought — “Did the AI solve the right problem?” is as important as “Is the code correct?”	
09:45	Designing the AI Code Review Checklist	75 min
	- EXERCISE 8: Build a comprehensive AI Code Review Checklist - Starting from your homework draft, expand into a structured, team-ready checklist: - Section 1 — Intent Verification: — Does the code solve the stated problem? (not a different, easier problem) — Does the implementation match the requirements/spec? — Are there unstated assumptions baked into the AI’s approach? - Section 2 — Correctness & Edge Cases: — Is the happy path correct? — Are error paths handled? (null, undefined, empty, boundary values) — Are race conditions possible? Is concurrency handled? - Section 3 — Security Scan: — Input validation? Output encoding? Authentication/authorization? — Secrets management? Dependency safety? — OWASP Top 10 quick check against the generated code	

	• Section 4 — Performance & Scalability: • — Time complexity? Space complexity? • — Database query efficiency? N+1 detection? • — Would this work at 10x current scale? • Section 5 — Maintainability & Standards: • — Naming conventions? Code style? Project patterns? • — Is this copy-paste from AI or adapted to the codebase? • — Would a new team member understand this without context? • Section 6 — Test Quality: • — Do tests actually verify behavior (not just execute code)? • — Are edge cases from Section 2 covered? • — Are tests independent, deterministic, and fast? • Format as a Markdown checklist in your repo — this will be used in all future PRs
--	--

11:00	Break	15 min
	• Stretch	

11:15	Hands-On: AI-Aware Code Review Practice	75 min
	• EXERCISE 9: Structured Code Review Practice • Instructor provides 4 pull requests, each containing AI-generated code for a realistic feature: • — PR #1: A user registration flow with email verification (Python/Flask) • — PR #2: A dashboard API with filtering and pagination (Node/Express) • — PR #3: A file processing pipeline with error handling (Python) • — PR #4: A WebSocket real-time notification system (TypeScript) • Each student reviews 2 PRs using their freshly built checklist • For each PR, leave structured review comments on GitHub: • — At least 1 “Approve” comment explaining what’s good and why • — At least 1 “Request Changes” comment with specific, actionable feedback • — At least 1 “Question” comment probing AI assumptions or unstated intent • After reviewing, compare your findings with the student who reviewed the same PR • Discussion: where did your checklist catch real issues? Where did it miss?	

12:30	Lunch	60 min
	• Optional: review the other 2 PRs you didn’t get to	

13:30	Automating Verification: CI/CD as Quality Gate	60 min
	• EXERCISE 10: Enhance your GitHub Actions pipeline with AI-aware quality gates: • — Add a code complexity check (e.g., Radon for Python, ESLint complexity rule for JS)	

	— Add a security scanning step (e.g., Bandit for Python, npm audit for Node) — Add a test coverage threshold that blocks merge below 80% — Add a PR template that includes the AI Code Review Checklist as a markdown checklist — (Advanced) Add a custom Action that flags files with high AI-generation markers - Key insight: the best verification is automated. Humans are for judgment; machines are for consistency. - Connect to the JD's Reliability responsibility: "Establish SLOs/SLIs across teams" — code quality SLOs start in CI
--	---

14:30	Break	15 min
	Stretch	

14:45	Team Workflow Design Workshop	75 min
	- EXERCISE 11: Design a team AI-augmented development workflow - In groups of 3–4, design a complete workflow for a team of 5 engineers using AI tools: — Who generates code? Who reviews? How does the review process differ from traditional reviews? — What's automated in CI/CD? What requires human judgment? — How do you handle disagreements between the AI's approach and team conventions? — How do you onboard new team members into this workflow? — How do you track AI-introduced defects and feed learnings back into prompts? - Each group presents their workflow design (5 min) to the class - Class votes on "Most Practical", "Most Innovative", "Most Scalable" - Key question: does your workflow make the team faster AND more reliable, or does it trade one for the other?	

16:00	Day 3 Wrap-Up	30 min
	- Publish your AI Code Review Checklist to your portfolio repo as a standalone Markdown file - Update your GitHub PR template to include the checklist - HOMEWORK: Review a real PR from another cohort member using your checklist — leave substantive comments - Preview: tomorrow we go deep on agent development — multi-tool architectures and memory systems	

Instructor Notes — Day 3: Day 3 is where the program starts developing the “train others” competency that distinguishes senior from mid-level. The checklist design exercise should feel like creating a real team artifact, not a homework assignment. Push students to make it specific enough to be useful but general enough to apply across projects. During the PR review exercise, watch for the quality of review comments — “LGTM” is not acceptable, and neither is

pure nitpicking. Coach toward the balance: flag what matters, explain why it matters, suggest how to fix it. The team workflow design is the day's capstone — it previews the organizational thinking required in Modules 7–8. Groups that produce a workflow diagram (not just bullet points) tend to think more systemically.

Day 4: Advanced Agent Development — Multi-Tool & Memory

Theme: Extend autonomous agents with multiple tools, persistent memory, and integration capabilities — building on the Nanobot foundation from Module 1

Goal: Every student has an enhanced Nanobot agent with 3+ functional tools, a persistent memory system, and a GitHub integration — deployed and tested via their CI/CD pipeline.

Schedule

09:00	Agent Architecture Patterns (Concept Session)	45 min
	- Recap from Module 1: Nanobot's basic architecture (message → agent → LLM → tools → response) - Three architectural patterns for agent enhancement: — Pattern 1: Multi-Tool Agent — multiple specialized tools the agent can invoke based on context — Pattern 2: Memory-Augmented Agent — persistent memory that evolves across conversations — Pattern 3: Integration Agent — connecting to external services (GitHub, APIs, databases) - The tool selection problem: how does an agent decide which tool to use? — System prompt guidance, function descriptions, and tool routing - Memory architecture: from flat conversation logs to structured knowledge — Short-term memory: current conversation context — Long-term memory: persistent facts, preferences, and project state (Markdown files in Nanobot) — Episodic memory: key events and decisions (the approach memU takes) - Security revisited: every new tool and integration is a new attack surface	
09:45	Build Sprint: Multi-Tool Agent	90 min
	- EXERCISE 12: Extend your Nanobot with a multi-tool architecture - Starting from your Module 1 fork, add at least 3 tools total: — Tool 1 (from Module 1): your existing custom skill — Tool 2 — GitHub Integration: query open issues, create issues, list recent commits - Use the GitHub API (or the gh CLI) from within your agent — Tool 3 — Choose one: - Option A: Web search tool (using a search API or web scraping) - Option B: File analysis tool (read and summarize local files or documents) - Option C: Code analysis tool (run linters or static analysis on code files and report results) - Option D: Calendar/scheduling tool (integrate with a calendar API)	

	• Use Claude Code CLI to help implement — but apply your Verification Lens and checklist to every generated line • Test each tool independently before integrating into the agent • Commit incrementally with descriptive messages — at least 5 commits during this sprint
--	--

11:15	Break	15 min
	• Troubleshoot, ask peers for help	

11:30	Build Sprint: Memory System	60 min
	• EXERCISE 13: Implement a structured memory system for your agent • Design a memory architecture that goes beyond flat conversation logs: • — User profile memory: preferences, recurring topics, expertise level • — Project memory: current tasks, decisions made, blockers identified • — Learning memory: corrections the user has made, mistakes to avoid • Storage: use Nanobot's local Markdown file system, but with structure: • — memory/profile.md — user facts and preferences • — memory/projects/ — per-project state files • — memory/corrections.md — things the agent got wrong and the user's fix • Test: have a multi-turn conversation, restart the agent, verify it remembers context • Key question: what should the agent remember, and what should it forget? (privacy by design)	

12:30	Lunch	60 min
	• Optional: keep building	

13:30	Build Sprint: Integration & CI/CD	90 min
	• EXERCISE 14: Wire everything together and deploy • — Integrate all 3 tools and the memory system into a cohesive agent • — Update the system prompt to give the agent clear guidance on when to use each tool • — Write tests for each tool (unit tests) and the integration (conversation flow tests) • — Update your GitHub Actions pipeline to: • → Lint the agent code • → Run tool unit tests • → Run integration tests (simulated conversations) • → Apply your AI Code Review Checklist as a PR template check • — Push as a PR to your Nanobot fork, request peer review • Apply your Day 2–3 skills: use the Verification Lens on all AI-generated code in this build	

15:00	Break	15 min
	Stretch	
15:15	Agent Showcase & Peer Review	45 min
	- EXERCISE 15: Agent Demo + Peer Code Review - Pair up: each person demos their agent to their partner (5 min each) - Show all 3 tools working - Show the memory system persisting across conversations - Show the CI/CD pipeline running - Then: each person reviews their partner's PR using the AI Code Review Checklist (15 min each) - Leave real review comments on GitHub - Focus especially on: tool selection logic, memory privacy, API key handling, error recovery	
16:00	Agent Architecture Retrospective	30 min
	- Class discussion: common patterns and problems across everyone's agents - How did agents handle ambiguous requests (when multiple tools could apply)? - What memory design decisions were most effective? - Where did AI-generated code cause the most issues in the agent build? - HOMEWORK: Address all code review comments from your peer. Merge the PR when ready. - HOMEWORK: Write a short architecture document for your enhanced agent (500 words + diagram) - Preview: tomorrow we formalize everything into an AI Governance framework	

Instructor Notes — Day 4: Day 4 is the build day, similar to Module 1 Day 4 but with much higher technical ambition. Students now have two weeks of experience plus the verification skills from Days 1–3. The GitHub Integration tool (Exercise 12, Tool 2) is mandatory because it connects the agent to the program's backbone and demonstrates practical value. The memory system exercise often surfaces interesting privacy discussions — what if the agent remembers something the user later wishes it hadn't? This connects to the AI Governance theme of Day 5. Watch for students who try to build all three tools with AI without reviewing — redirect them to their own checklist. The peer review at end of day should be substantive; if students are rubber-stamping, pause and coach on review quality.

Day 5: AI Governance Lab, Reflection & Calibration

Theme: Synthesize the week's learning into an AI Governance framework, update your Personal Development Map, and prepare for Module 3

Goal: Every student has a publishable AI Governance framework, has completed all deliverables, and can articulate a clear philosophy on AI-augmented development.

Schedule

09:00	AI Governance Framework Design (Workshop)	90 min
	- EXERCISE 16: Create your AI Governance Framework for Engineering Teams - This is the capstone synthesis of Module 2 — combining prompt engineering, bug detection, code review, and agent development into a coherent framework: - Part 1 — Principles (what we believe): — Write 5–7 principles for AI-augmented development (e.g., “AI output is a draft, never a final product”) — Draw from your personal experience this week and the Bug Log evidence - Part 2 — Workflows (how we work): — Define the workflow from prompt → generation → verification → merge — Include your AI Code Review Checklist as the team standard — Specify when human-in-the-loop review is mandatory vs. optional - Part 3 — Tooling (what we use and when): — Tool selection guide: Claude Code for X, Kiro for Y, Antigravity for Z — CI/CD quality gates: which automated checks are required? - Part 4 — Training (how we onboard): — What should a new team member learn in their first week of AI-augmented development? — How should they use the Bug Log as a training resource? — What exercises would build their Verification Lens? - Part 5 — Measurement (how we know it's working): — How do you track AI-introduced defects? — How do you measure whether AI tools are making the team faster AND more reliable? — What are the red flags that AI is hurting more than helping? - Format as a Markdown document in your portfolio repo	
10:30	Break	15 min
	Refresh	
10:45	Framework Presentations & Peer Feedback	60 min

	• EXERCISE 17: Present your governance framework to a small group (3–4 people, 7 min each) • — Focus on: what's unique in your framework? What principles did this week's experiences drive? • — Peer feedback: what would you adopt from this framework? What's missing? • — Each person takes notes on feedback to refine their framework afterward • Best practice: the best governance frameworks are opinionated but evidence-based — cite your Bug Log
--	--

11:45	Lunch	60 min
	• Form study groups	

12:45	Personal Development Map Revisit	45 min
	• EXERCISE 18: Update your Personal Development Map (recurring ritual) • — Re-rate yourself on the five mindsets — has Precision or Systems thinking grown? • — On the four competency layers: Module 2 targeted Layer 2 (Architect) and Layer 3 (Enabler) • — New gaps discovered? (Common: students realize prompt engineering is a deeper skill than expected) • — Refine your 10-week intention based on 2 weeks of experience • Instructor 1:1s (5 min each): progress check, concerns, preview of Module 3 readiness	

13:30	Impossible Stuff Day: Self-Directed Exploration	120 min
	• Week 2 Impossible Stuff Day suggestions: • — Add a 4th or 5th tool to your agent (browser control, email, smart home, etc.) • — Experiment with different LLMs in Nanobot (swap Claude for Gemini or a local Llama model) and compare agent behavior • — Build a “prompt optimizer” that takes a vague prompt and rewrites it using the 5-layer structure • — Create an automated Bug Safari: a script that generates AI code and tests it for common failure patterns • — Read ahead: skim the NanoClaw codebase for Module 4 prep • — Write a blog post: “What I Learned About Trusting AI-Generated Code” • Share explorations in the last 15 min	

15:30	Break	15 min
	• Stretch	

15:45	Portfolio Polish & Module 3 Preview	30 min
--------------	--	--------

	• EXERCISE 19: Deliverables quality check (see Appendix B checklist) • Module 3 preview: Full-Stack Development at Speed • — Next week you build a complete full-stack application in one week using AI tools • — Antigravity's multi-agent Manager View for parallelizing development • — Advanced CI/CD: Docker, multi-environment deployment, matrix builds • Pre-work for Module 3: • — Generate a NotebookLM Audio Overview for Module 3 material • — Choose a full-stack framework you want to learn or deepen (React + Node, Python + FastAPI + React, etc.) • — Think about what application you'd build if you had a full week
--	---

16:15	Badge Ceremony & Week 2 Close	30 min
	• Badge ceremony: AI-Augmented Developer badge for students who completed all deliverables • Call out achievements: best Bug Log entry, most comprehensive checklist, most creative agent tool, best governance framework • Read the Developer Manifesto together (recurring ritual) • Reflection: "The most important thing I learned about working with AI this week is..." • Week 1 was about learning to build with AI. Week 2 was about learning to verify. Week 3 is about learning to build FAST.	

Instructor Notes — Day 5: The Governance Framework exercise (Exercise 16) is the week's integration moment. It should feel like creating a real artifact for a real team, not an academic exercise. Push students to ground every principle in specific evidence from their Bug Log — "We believe X because we observed Y." The framework presentations should feel like a team lead presenting to their engineering org. Coach students who present too casually or too academically. The Development Map revisit should show real growth — if students rate themselves the same as Week 1, probe whether they're self-reflecting deeply enough. The badge ceremony should now feel like an established ritual. Build on it each week.

Appendix A: Exercise Summary

#	Day	Exercise	Type
1	1	Prompt Playbook: 5 systematic patterns (scaffold, test, refactor, debug, document)	Workshop
2	1	Single-shot vs. chain-of-thought vs. decomposition prompt comparison	Analysis
3	1	Prompt Portability Test: same prompts across Claude Code, Kiro, Antigravity	Comparative
4	1	Pair prompt review: exchange and test partner's playbook	Collaborative
5	2	AI Bug Safari Round 1: review 10 curated samples with planted bugs	Detective
6	2	AI Bug Safari Round 2: generate code for 4 tasks, find wild bugs	Hands-on
7	2	Compile AI Bug Log with structured entries (10+ bugs)	Documentation
8	3	Design AI Code Review Checklist (6 sections)	Framework
9	3	Structured code review practice on 2 instructor-provided PRs	Practice
10	3	Enhance CI/CD with automated quality gates	Hands-on
11	3	Team workflow design for AI-augmented development	Design
12	4	Extend Nanobot with 3+ tools (incl. GitHub integration)	Build
13	4	Implement structured memory system for agent	Build
14	4	Integration, testing, CI/CD pipeline, and PR submission	Build
15	4	Agent demo + peer code review using AI checklist	Collaborative
16	5	AI Governance Framework (principles, workflows, tooling, training, measurement)	Synthesis
17	5	Framework presentation + peer feedback	Presentation
18	5	Personal Development Map revisit and update	Reflection
19	5	Deliverables quality check and portfolio polish	Quality

Appendix B: Deliverables Checklist

Complete all items to earn the AI-Augmented Developer badge:

1. **Prompt Playbook** — 15+ organized prompt patterns with examples, including chain-of-thought and decomposition patterns, in prompts/ directory
2. **AI Bug Log** — 12+ documented AI code failures, each with: buggy code, category, root cause, detection method, fix, prevention strategy
3. **AI Code Review Checklist** — 6-section team-ready checklist (Intent, Correctness, Security, Performance, Maintainability, Tests) published as standalone Markdown
4. **Enhanced Nanobot Agent** — Forked repo with 3+ tools (including GitHub integration), structured memory system, passing CI/CD pipeline, peer-reviewed PR
5. **AI Governance Framework** — 5-part document (Principles, Workflows, Tooling, Training, Measurement) with evidence from Bug Log, published in portfolio repo
6. **Peer Reviews** — At least 2 substantive code reviews given to peers using the AI Code Review Checklist, with constructive comments on GitHub
7. **Personal Development Map v2** — Updated self-assessment showing growth from Module 1, with specific evidence and refined 10-week intention
8. **GitHub Portfolio** — All deliverables committed, CI/CD pipeline green, Projects board updated with Module 2 issues tracked and closed

Appendix C: AI Bug Safari — Instructor Preparation Guide

For instructors: The 10 curated code samples for the Bug Safari (Exercise 5) must be prepared in advance. This appendix provides guidance on creating realistic, pedagogically effective samples.

Sample Preparation Principles

1. **Generate, don't write:** Use actual AI tools to generate the base code, then selectively plant additional bugs. This ensures the code “feels” AI-generated (consistent style, plausible patterns) rather than hand-crafted to be broken.
2. **One primary bug, 1–2 secondary:** Each sample should have one main learning objective (the primary bug) and optionally 1–2 smaller issues. Too many bugs per sample is overwhelming; too few is not challenging enough.
3. **Vary languages:** Use a mix of Python, JavaScript/TypeScript, and at least one other language (Go, Rust, or Java) to reflect the polymath expectation.
4. **Include context:** Each sample should come with a brief requirements description and the prompt that was used to generate it. This lets students practice intent verification (Section 1 of the checklist).
5. **Progressive difficulty:** Samples 1–3 should be catchable by most mid-level engineers. Samples 4–7 should require careful reading. Samples 8–10 should be genuinely tricky — the kind of bugs that make it to production.

Bug Category Distribution

Recommended distribution across the 10 samples to ensure full taxonomy coverage:

Category	Samples	Why This Weight
Confident Hallucination	1	Easiest to spot; serves as a warm-up
Subtle Logic Errors	3	Most common AI failure in practice; multiple examples build pattern recognition
Security Blind Spots	2	Critical for the Reliability responsibility; often missed on first pass
Performance Anti-Patterns	2	Invisible without analysis; teaches Big-O thinking and load awareness
Stale Knowledge	1	Common but less dangerous; teaches checking API currency
Copy Without Context	1	Connects to the 4x code clone industry data; teaches adaptation thinking

Answer Key: Prepare a detailed answer key for each sample with: (a) the exact bugs, (b) the

category and why, (c) the line numbers, (d) the ideal detection method, and (e) the fix. This is used during the 11:30 AM debrief session. Share the answer key with students AFTER the exercise, not before.

Cohort Bug Repository: Create a shared GitHub repository for the cohort's collective Bug Log. After Module 2, encourage students to contribute new bugs they discover throughout the rest of the program. This becomes an increasingly valuable training resource for future cohorts.

End of Module 2 Lesson Plan